SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192311366**
Name	**GONDEL PRSHANTH**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

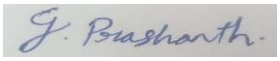
Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I Gondel Prashanth / 192311366 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Answers for above questions

1) AI Search Algorithms Case Study: Emergency Medicine Delivery by Drone

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) selects the next node based only on the heuristic value h(n), which estimates the remaining distance to the destination.

Evaluation Function:

$f(n)=h(n)$

In the flood-affected region, the drone always chooses the path that appears closest to the emergency location based on the straight-line distance. It does not consider the actual travel cost or delays caused by flooded roads, strong winds, or difficult terrain. If a chosen route becomes blocked, the drone must search for another path.

Strengths

- Makes decisions very quickly.
- Requires less computation and memory.
- Suitable when an immediate response is needed.
- Efficient if the heuristic estimate is accurate and the environment is stable.

Weaknesses under Uncertainty

- Ignores the actual travel cost from the starting point.
- May choose risky or blocked routes because it only considers estimated distance.
- Does not guarantee the shortest or safest path.
- Frequent replanning is required when real-time updates indicate blocked paths.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

A* Search considers both the actual travel cost and the estimated remaining cost to reach the goal.

Evaluation Function:

$f(n)=g(n)+h(n)$

Where:

- g(n) = Actual cost from the start node.
- h(n) = Estimated cost to reach the goal.

Behavior in the Scenario

The drone evaluates every possible route by combining the cost already traveled with the estimated distance to the destination. If heavy rain, flooding, or strong winds increase the travel

cost, A* automatically prefers safer or less expensive alternative routes. This helps the drone avoid dangerous or highly delayed paths.

Advantages

- Finds the optimal route when the heuristic is admissible.
- Considers both travel cost and estimated distance.
- Avoids expensive or risky routes.
- Produces more reliable results in changing environments.

Limitations

- Requires more memory than Greedy Best-First Search.
- Performs more computations.
- May need to replan the route whenever new obstacles appear.

iii. Analyze the impact of heuristic accuracy on both algorithms

The quality of the heuristic has a significant effect on the performance of both algorithms.

Impact on Greedy Best-First Search

If the heuristic is highly accurate, the drone quickly reaches the destination with minimal searching. However, if the heuristic is inaccurate, the drone may repeatedly move toward blocked or unsafe routes, resulting in delays and inefficient navigation. Since GBFS depends entirely on the heuristic, its performance decreases greatly when the estimate is poor.

Impact on A*

A* also benefits from an accurate heuristic because it expands fewer nodes and reaches the destination more efficiently. Even if the heuristic is less accurate, A* still considers the actual travel cost, making it more reliable than GBFS. When an admissible heuristic is used, A* continues to guarantee the optimal solution.

iv. Discuss how dynamic changes (blocked paths) affect search performance

Flood conditions are highly dynamic, and roads or air routes may become blocked unexpectedly.

In Greedy Best-First Search, the drone may continue moving toward a blocked path because it only follows the heuristic estimate. When a blockage is detected, the algorithm often has to restart or perform significant replanning, increasing travel time.

A* handles dynamic changes more effectively by recalculating the route using updated movement costs and obstacle information. Although replanning increases computational effort, the drone is more likely to find a safe and efficient alternative path. Therefore, A* performs better in environments with frequent changes.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Although Greedy Best-First Search makes faster decisions, it ignores the actual travel cost and may select blocked or unsafe routes. In a flood-affected region where travel costs and obstacles change continuously, this can lead to delays and increased risk.

A* balances the actual travel cost with the heuristic estimate, allowing the drone to choose routes that are both efficient and safe. It adapts better to changing weather conditions, blocked paths, and varying terrain costs while still finding an optimal route when an admissible heuristic is used.

2) AI Optimization Case Study: Smart City Traffic Signal Optimization

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing is a local search algorithm that starts with an initial traffic signal schedule and evaluates its performance. It then makes small changes, such as increasing or decreasing the green signal duration at one or more intersections. If the new schedule improves traffic flow by reducing waiting time or congestion, the system accepts the change. This process continues until no neighboring solution provides a better result.

Advantages

- Simple and easy to implement.
- Requires less memory.
- Quickly finds improved solutions.
- Suitable for continuous optimization problems.

Limitations

- May stop at a local optimum instead of finding the best overall solution.
- Cannot easily escape from poor-quality solutions.
- Performance depends heavily on the initial traffic signal settings.
- Less effective in highly dynamic traffic environments.

ii. Discuss Issues like Local Maxima, Plateaus, and Ridges with Real-World Interpretation

Local Maxima

A local maximum is a solution that is better than its neighboring solutions but is not the best possible solution.

Real-world interpretation:

The AI finds a traffic signal timing that reduces congestion in one part of the city. However, another timing configuration elsewhere could reduce congestion much more, but the algorithm cannot reach it because all nearby changes appear worse.

Plateaus

A plateau is a flat region where many neighboring solutions have the same evaluation value.

Real-world interpretation:

Several different traffic signal timings produce almost identical waiting times. Since no immediate improvement is visible, the algorithm cannot determine which direction to move and may stop or wander randomly.

Ridges

A ridge is a narrow path toward the optimal solution that requires multiple coordinated changes rather than a single small adjustment.

Real-world interpretation:

Improving traffic flow may require changing signal timings at several connected intersections simultaneously. Changing only one intersection does not improve traffic, so Hill Climbing fails to discover the better solution.

iii. Explain how Simulated Annealing or Genetic Algorithms can Overcome These Limitations

Simulated Annealing

Simulated Annealing improves upon Hill Climbing by occasionally accepting worse solutions during the early stages of the search. This allows the algorithm to escape local optima and continue exploring the search space. As the search progresses, the probability of accepting worse solutions gradually decreases until the algorithm focuses on finding the best solution.

Advantages

- Escapes local maxima.
- Handles complex optimization problems.
- Suitable for dynamic environments.
- Produces better-quality solutions than Hill Climbing.

Genetic Algorithms

Genetic Algorithms are inspired by natural evolution. Each traffic signal configuration is treated as an individual in a population. The algorithm evaluates each solution, selects the best-performing ones, combines their characteristics through crossover, and introduces random mutations to create new solutions. This process repeats over many generations until an effective traffic signal plan is found.

Advantages

- Explores many solutions simultaneously.
- Avoids getting trapped in local optima.
- Highly scalable for large search spaces.
- Adapts well to changing traffic conditions.
- Produces near-optimal solutions for complex optimization problems.

iv. Recommend the Best Approach and Justify Your Choice Based on Scalability and Adaptability

Genetic Algorithm is the most suitable approach for this scenario.

The traffic optimization problem involves hundreds of intersections and a very large search space. Traffic conditions also change continuously throughout the day. Hill Climbing is fast but often becomes trapped in local optima, making it unsuitable for such complex environments. Simulated Annealing performs better because it can escape local optima, but it still explores one solution at a time.

Genetic Algorithms evaluate multiple traffic signal configurations simultaneously and continuously improve them through selection, crossover, and mutation. They are highly scalable, adapt well to changing traffic patterns, and can discover high-quality solutions even in complex environments. Therefore, Genetic Algorithms provide the best balance between optimization quality, scalability, and adaptability for smart city traffic signal optimization.

3) AI Search Case Study: Autonomous Mars Rover

i. Explain why this problem requires an online search agent instead of offline search

An online search agent is required because the Mars rover operates in an unknown environment where it does not have a complete map before starting its mission. Unlike offline search, which assumes that the entire environment is known in advance, the rover discovers new information only as it moves.

As the rover explores, it senses nearby terrain, identifies obstacles such as rocks and craters, and updates its path accordingly. It must make decisions in real time based on the information currently available. Therefore, an online search agent is more suitable because it can plan and act simultaneously while continuously learning about the environment.

ii. Analyze the challenges of partial observability and dynamic environments

Partial Observability

The rover has limited sensing capability, meaning it can observe only the nearby area instead of the entire Martian surface. Hidden obstacles or hazards may exist beyond the sensor range, making it difficult to determine the safest route.

Challenges

- Incomplete information about the environment.
- Hidden obstacles may suddenly appear.
- Difficult to predict the best path.
- Frequent updates are required as new information becomes available.

Dynamic Environment

Although the Martian terrain changes slowly, new hazards such as loose rocks, dust movement, or sensor inaccuracies can affect navigation. As the rover explores, newly detected obstacles may require it to change its planned route.

Challenges

- Previously safe paths may become unsafe.
- Continuous replanning is required.
- Increased computational effort for real-time decision-making.
- Higher risk if changes are not detected quickly.

iii. Describe how the rover builds and updates its internal model

The rover maintains an internal model of its surroundings based on the information collected through its sensors.

Initially, the internal model contains very little information. As the rover moves, it scans the nearby terrain and records the locations of rocks, craters, slopes, and safe paths. This newly discovered information is stored in its internal map.

Whenever new obstacles or safer routes are detected, the rover updates its internal model and modifies its navigation plan. Over time, the internal model becomes more accurate, enabling the rover to make safer and more efficient decisions while exploring Mars.

iv. Compare online search with uninformed and informed search strategies

Online Search

Online search explores the environment while simultaneously making decisions. It does not require complete knowledge of the terrain before starting and continuously updates its plan using newly discovered information. It is highly suitable for unknown environments like Mars.

Uninformed Search

Uninformed search algorithms, such as Breadth-First Search (BFS) and Depth-First Search (DFS), do not use heuristic information. They assume that the environment is already known and systematically explore possible paths. These algorithms are inefficient for the Mars rover because they require a predefined map and may explore many unnecessary paths.

Informed Search

Informed search algorithms, such as Greedy Best-First Search and A* Search, use heuristic information to guide the search toward the goal more efficiently. They perform well when a reasonably accurate map or heuristic is available. However, in an unknown environment, their effectiveness is limited because the heuristic may become inaccurate as new terrain is discovered.

Compared with uninformed and informed search, online search is more appropriate because it combines exploration with decision-making and continuously adapts to newly available information.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

An Online Search Agent is the most suitable approach for this scenario.

The Mars rover operates in an environment where the terrain is unknown and only partially observable. Since complete information is unavailable before exploration, offline search methods cannot produce an effective plan. The rover must continuously observe its surroundings, update its internal model, and revise its route whenever new hazards are detected.

An online search agent provides high adaptability by making real-time decisions based on current sensor information. It improves safety by avoiding newly discovered obstacles and selecting safer paths as exploration continues. Although online search may require more

computation due to continuous replanning, its ability to operate under uncertainty and respond to changing conditions makes it the best choice for autonomous Mars exploration.

4) AI Case Study: University Exam Timetable Generation Using Constraint Satisfaction Problem (CSP)

i. Clearly Define Variables, Domains, and Constraints with Explanation

Variables

Variables represent the exams that need to be scheduled.

Examples:

- Exam for Mathematics
- Exam for Physics
- Exam for Chemistry
- Exam for Computer Science
- Exam for English

Each exam is treated as a separate variable whose schedule must be determined.

Domains

The domain represents the possible values that each variable can take.

For each exam, the domain includes:

- Available dates
- Available time slots
- Available examination halls

Example:

- Mathematics → Monday Morning, Hall A
- Mathematics → Tuesday Afternoon, Hall B
- Mathematics → Wednesday Morning, Hall C

The AI selects one valid combination for each exam.

Constraints

The timetable must satisfy all of the following constraints:

- No student should have overlapping examinations.
- The number of available examination halls cannot be exceeded.
- Some subjects must be scheduled before others according to academic requirements.
- Faculty members must be available during the assigned examination time.
- Students requiring special accommodations must be assigned suitable halls and facilities.
- Each examination hall can host only one examination at a given time.
- Every exam must be assigned exactly one valid time slot and hall.

A timetable is considered valid only if all these constraints are satisfied.

ii. Explain how Backtracking Search Works in this Scenario

Backtracking Search is a systematic algorithm used to solve CSPs by assigning values to variables one at a time.

Initially, the AI selects an exam and assigns it an available time slot and examination hall. It then proceeds to schedule the next exam while checking whether the current assignment violates any constraints.

If a conflict occurs, such as two exams being scheduled for the same students at the same time or exceeding the available exam halls, the algorithm rejects that assignment. It then returns to the previous exam and tries another possible time slot or hall.

This process continues until either:

- A complete timetable satisfying all constraints is found, or
- All possible assignments have been explored.

Backtracking guarantees that if a valid timetable exists, it will eventually find one.

iii. Discuss how Constraint Propagation (e.g., Forward Checking) Improves Efficiency

Constraint propagation reduces the search space by eliminating invalid choices before they lead to conflicts.

One common technique is Forward Checking.

When an exam is assigned to a particular time slot and hall, Forward Checking immediately removes conflicting values from the domains of the remaining unscheduled exams.

For example, if Mathematics is scheduled on Monday Morning in Hall A:

- Other exams using Hall A at the same time are removed from consideration.
- Exams taken by the same students cannot be scheduled during that time slot.
- Time slots violating faculty availability are also eliminated.

By removing impossible choices early, Forward Checking:

- Detects conflicts sooner.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.
- Improves overall efficiency, especially for large universities.

iv. Analyze Real-World Challenges and Justify the Best Strategy for Scalability

Real-World Challenges

Generating an examination timetable becomes increasingly difficult as the university grows.

Some major challenges include:

- Large numbers of courses and students increase the search space.
- Numerous constraints must be satisfied simultaneously.
- Faculty availability may change unexpectedly.
- Limited examination halls create scheduling conflicts.
- Students with special accommodations require additional scheduling considerations.
- Last-minute changes, such as adding or postponing exams, require rapid timetable updates.

These factors make manual scheduling extremely difficult and time-consuming.

Best Strategy for Scalability

The most effective strategy is to combine Backtracking Search with Constraint Propagation (Forward Checking).

Backtracking ensures that all constraints are satisfied by systematically exploring possible schedules. Forward Checking improves efficiency by removing invalid choices as soon as assignments are made, greatly reducing unnecessary searches.

For very large universities with hundreds of courses and thousands of students, additional CSP heuristics such as selecting the most constrained variable first or choosing the least constraining value can further improve performance. Together, these techniques allow the AI system to generate high-quality exam timetables efficiently while adapting to changing requirements.

5) AI Case Study: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

i. Explain how the Minimax Algorithm Models this Problem

The Minimax algorithm is a decision-making algorithm used in two-player competitive games. It assumes that one player (the AI) tries to maximize its chances of winning, while the opponent (the human player) tries to minimize the AI's chances of winning.
In this strategy game, every game state is represented as a node in a game tree, and each possible move creates a new branch. The AI explores the possible future moves of both players before selecting its next action.
The algorithm works as follows:

- The AI (MAX player) chooses the move that gives the highest possible score.
- The human opponent (MIN player) chooses the move that gives the lowest score for the AI.
- The AI assumes that the opponent always plays optimally.
- The search continues until a predefined depth or the end of the game is reached.
- The AI selects the move with the highest Minimax value.

This enables the AI to anticipate the opponent's actions and choose the best possible strategy.

ii. Analyze the Computational Challenges of Large Game Trees

Real-time strategy games have an enormous number of possible game states.
The major computational challenges include:

- Every move creates many possible future moves, causing the search tree to grow exponentially.
- Exploring every possible move requires a large amount of computation.
- Memory usage increases significantly as the game tree becomes deeper.
- Real-time games require decisions within a few milliseconds, making exhaustive search impractical.
- Multiple units, resources, and strategic options further increase the complexity of the search space.

As the game progresses, evaluating every possible move becomes impossible within the available time.

iii. Explain how Alpha-Beta Pruning Improves Efficiency with Detailed Reasoning

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It reduces the number of nodes that need to be evaluated without affecting the final decision.
The algorithm maintains two values:

- Alpha (α): The best value that the MAX player can guarantee.
- Beta (β): The best value that the MIN player can guarantee.

During the search:

- If the algorithm finds that a branch cannot produce a better result than the current best option, that branch is ignored.
- Branches that cannot influence the final decision are pruned.
- Only promising branches are explored further.

Advantages of Alpha-Beta Pruning

- Reduces the number of nodes evaluated.
- Speeds up the search process.
- Allows deeper game-tree exploration within the same time limit.
- Produces the same optimal move as the standard Minimax algorithm.
- Makes real-time decision-making more practical.

Thus, Alpha-Beta Pruning significantly improves the efficiency of Minimax while maintaining the quality of decisions.

iv. Design an Evaluation Function and Justify the Factors Considered

When the search cannot reach the end of the game, the AI uses an evaluation function to estimate how favorable a game state is.
A possible evaluation function is:

$$\text{Evaluation Score} = (5 \times \text{Army Strength}) + (4 \times \text{Resources}) + (3 \times \text{Map Control}) + (2 \times \text{Unit Health}) - (5 \times \text{Enemy Strength})$$

Factors Considered
Army Strength

- Number and power of military units.
- Stronger armies increase the score.

Resources

- Amount of gold, food, energy, or other resources available.
- More resources allow future expansion.

Map Control

- Control of important strategic locations.
- Greater control provides tactical advantages.

Unit Health

- Health of friendly units.
- Healthier units improve combat effectiveness.

Enemy Strength

- Number and power of opponent units.
- Stronger enemy forces reduce the evaluation score.

The evaluation function estimates the quality of a game state when the complete search is not possible.

v. Recommend a Strategy for Balancing Optimality and Real-Time Performance
The most effective strategy is to combine Minimax with Alpha-Beta Pruning and use a depth-limited search guided by an evaluation function.
In this approach:

- Minimax selects the best possible move by considering both the AI's and the opponent's actions.
- Alpha-Beta Pruning eliminates unnecessary branches, reducing computation time.
- A depth limit ensures that decisions are made within the available time.
- The evaluation function estimates the quality of non-terminal game states.
- The AI can continuously update its decisions as the game progresses.

This combination provides high-quality decisions while satisfying the strict timing requirements of real-time strategy games.